

최적화개론 Project 2

2025-1학기

수업번호 10021



Group 5

미래자동차공학과 2019048440 최윤석

전기공학전공 2020045441 문재욱

1. 서론

Knapsack 문제는 제한된 자원 내에서 최대의 이익을 추구하는 조합 최적화 문제로, 다양한 알고리즘 설계 기법의 성능을 비교·분석하는 데 이상적인 실습 사례이다. 본 프로젝트는 이러한 Knapsack 문제를 대상으로 하여 여러 알고리즘의 해법을 구현하고, 그 결과를 바탕으로 성능을 비교·분석하는 것을 주요 목표로 한다.

Problem 1에서는 동적 프로그래밍(Dynamic Programming)을 활용하여 최적 비용뿐만 아니라 해당 최적 해를 구성하는 아이템 조합(최적 전략)까지 도출할 수 있는 알고리즘을 구현한다. 이는 단순히 최적값만 구하는 것이 아니라, 실제 의사결정에 적용 가능한 해를 제시한다는 점에서 중요하다.

Problem 2에서는 분기 한정법(Branch and Bound)을 학습하여, 탐색 공간을 체계적으로 축소하며 최적 해를 탐색하는 방식의 장단점을 파악한다.

이와 더불어, 탐욕 알고리즘(Greedy Algorithm), 선형계획법(Linear Programming, LP), LP 해의 반내림 기반 알고리즘(LP Round Down)과도 비교하고자 한다. 이들 알고리즘은 계산 속도나 구현 용이성 측면에서 장점이 있지만, 항상 최적 해를 보장하지는 않는다. 따라서 본 프로젝트에서는 각 알고리즘의 시간 복잡도, 해의 정확도, 구현 난이도 등의 측면에서 성능을 비교 분석하며, 문제 유형에 따라 어떤 알고리즘이 상대적으로 더 적합한지를 살펴볼 것이다.

이와 같은 분석을 통해 Knapsack 문제를 해결하는 다양한 접근 방식의 특성과 한계를 이해하고, 실제 문제 해결에 있어 적절한 알고리즘 선택 능력을 기르는 것을 본 프로젝트의 주요 목표로 설정하였다.

****문제 조건 정의****

p_j , w_j and c are positive integers,

$$\sum_{j=1}^n w_j > c,$$

$$w_j \leq c \text{ for } j \in N.$$

$$\frac{p_1}{w_1} \geq \frac{p_2}{w_2} \geq \dots \geq \frac{p_n}{w_n}.$$

knapsack 문제를 해결하기 위해서는 다음 4가지의 전제 조건이 필요하다.

- 1) 각 물건의 무게와 가치, 가방의 용량은 양의 정수로 정의된다
- 2) 각 물건의 무게 합은 가방의 용량보다 커야 한다.
(무게의 합이 가방 용량보다 작으면 모든 물건을 선택하여 가져가면 되기 때문이다.)
- 3) 각 물건의 무게는 가방의 용량보다 클 수 없다.
(물건 하나의 무게가 가방 용량보다 크다면, 선택하지 않으면 되기 때문이다.)
- 4) 각 물건의 무게와 가치 값에 대해서, $\frac{\text{가치}}{\text{무게}}$ 값이 큰 순서부터 나열해야 한다.

문제를 해결하는 과정에서 조건에 부합하는 값들이 주어졌을 때 코드 실행을 용이하게 하고, 한 번에 비교할 수 있도록 물건에 대한 정보와 가방 용량은 따로 정의되도록 설계하였으며, 해를 구할 수 있는 각 방법들은 함수로 정의하여 결과를 도출하도록 하였다.

문제 해결에 필요한 수치 정의

```
import numpy as np
from scipy.optimize import linprog
import time

np.random.seed(15)

# 1. 설정 (값들은 모두 정수를 가지도록 설정)
size = 10 # 아이템 수
W = np.random.randint(5, 50) # 배낭 최대 무게

# 적절히 작은 무게로 제한 (1 ~ W//3)
while True:
    # 무게가 W를 넘지 않도록 설정
    weights = np.random.randint(1, W // 3, size=size)
    # 무게의 총 합은 W를 초과하도록 설정
    if weights.sum() > W and all(w < W for w in weights):
        break

# 가치도 동일한 사이즈로 생성
values = np.random.randint(5, 10, size=size)

# 출력
print("weights =", list(weights))
print("values =", list(values))
print("capacity =", W) # 가방에 들어갈 수 있는 무게

weights = [2, 1, 2, 1, 1, 2, 3, 2, 2, 2]
values = [5, 7, 9, 6, 8, 9, 9, 5, 7, 6]
capacity = 13
```

우선, 랜덤한 값들을 사용할 때 그 특성상 결과를 재현하기 어렵기 때문에 시드를 사용하여 같은 결과를 도출할 수 있도록 하였다.

이후 물건의 개수와 배낭 무게를 설정하고, 물건의 각 무게에 대한 조건을 제시하여 값을 생성한다. 이 때 while을 사용하여 가방의 무게보다 적당히 작은 물건 무게를 생성하도록 하였고, if문을 사용하여 모든 물건의 무게 합은 가방의 용량보다 커지도록 설정하였다. 가치도 적당한 크기의 값을 할당한 후, 랜덤으로 지정된 값을 확인하기 위해 print 문으로 값 할당 결과를 확인한다.

```
# 2. 아이템 정보 결함 (무게, 가치, 원래 인덱스 1부터 시작)
items = list(zip(weights, values, range(1, len(weights) + 1))) [정렬된 아이템 목록] (번호, 무게, 가치):

# 3. 정렬 (가치/무게 비율 기준, 내림차순)
n = len(items)
for i in range(n):
    # 한 요소에 대해서 모든 값들과 대소비교하여 자리 조정
    for j in range(0, n - i - 1):
        ratio1 = items[j][1] / items[j][0]
        ratio2 = items[j + 1][1] / items[j + 1][0]
        if ratio1 < ratio2:
            items[j], items[j + 1] = items[j + 1], items[j]

print("\n[정렬된 아이템 목록] (번호, 무게, 가치):")
for idx, (w, v, original_index) in enumerate(items, start=1):
    print(f"{idx}. ({w}, {v})")
```

1. (1, 8)
2. (1, 7)
3. (1, 6)
4. (2, 9)
5. (2, 9)
6. (2, 7)
7. (3, 9)
8. (2, 6)
9. (2, 5)
10. (2, 5)

앞서 할당된 무게와 가치의 값들을 연동하여 하나의 리스트에 저장한다. 이 과정은 각 물건의 무게 정보와 가치 정보를 동시에 파악할 수 있게 해준다.

이후 4) 조건을 만족시키기 위해 각 물건의 무게와 가치 값에 대해서, $\frac{\text{가치}}{\text{무게}}$ 값이 큰 순서부터 나열한다. 정렬된 물건 목록에 물건을 하나씩 추가할 때마다 모든 요소와의 대소 비교를 통해 스위칭하며 값 내림차순으로 정렬한다. 앞서 무게와 가치를 연동하는 과정을 거쳤기 때문에 무게와 가치의 위치가 동시에 스위칭된다.

실행 결과를 확인하기 위해 print문을 작성하였으며, 편의를 위해 인덱스 번호까지 할당하였다.

2. 동적 프로그래밍(Dynamic Programming)

동적 프로그래밍은 문제를 여러 개의 더 간단한 하위 문제로 나누어 해결하는 알고리즘이다. Greedy 알고리즘과 달리 모든 하위 문제를 재귀적으로 해결해야 하므로 더 복잡하고 시간 복잡도가 느리며 이전 하위 문제 해를 저장하는 데 더 많은 메모리가 필요하다. Greedy 알고리즘은 문제에 따라

최적의 해를 얻을 수 있다고 보장할 수 없는 반면, 동적 프로그래밍은 최적의 해를 확실하게 계산할 수 있다는 장점이 있다.

<Python Code 분석>

```
242 # 4. DP를 이용한 0/1 Knapsack 알고리즘
243 def Dynamic():
244     start_time=time.time()
245     global items
246     dp = [[0] * (W + 1) for _ in range(n + 1)] # dp[i][w] = i번째까지 고려, 무게 w일 때 최대 가치
247
248     for i in range(1, n + 1):
249         weight = items[i - 1][0]
250         value = items[i - 1][1]
251         for w in range(W + 1):
252             if weight <= w:
253                 dp[i][w] = max(dp[i - 1][w], dp[i - 1][w - weight] + value)
254             else:
255                 dp[i][w] = dp[i - 1][w]
```

(1) DP를 이용한 0/1 Knapsack 알고리즘

1) start_time = time.time() : 함수 시작 시간 기록. 전체 알고리즘의 실행 시간을 측정하는 데 활용.

2) global items: 전역 변수 'items'를 사용하겠다고 선언하는 것. items는 (weight, value)쌍으로 구성된 아이템 리스트임.

3) dp = [[0] * (W + 1) for _ in range(n + 1)]: DP 테이블 초기화.
'dp[i][w]'는 i번째 아이템까지 고려했을 때, 무게 한도가 w일 때 얻을 수 있는 최대 가치를 저장함.

4) for i in range(1, n + 1): 1번 아이템부터 n번 아이템까지 반복. 각 아이템에 대해 DP 값을 계산함.

5) weight = items[i - 1][0]
value = items[i - 1][1]

i번째 아이템의 무게와 가치를 추출

6) for w in range(W + 1): 배낭의 용량 w를 0부터 W까지 반복하며 해당 무게에서의 최대 가치를 계산

7) if weight <= w: 현재 아이템의 무게가 w보다 작거나 같다면, 해당 아이템을 넣을 수 있는 경우

8) $dp[i][w] = \max(dp[i - 1][w], dp[i - 1][w - \text{weight}] + \text{value})$

: (1) 현재 아이템을 넣지 않았을 때의 최대 가치 $dp[i - 1][w]$ 와 (2) 현재 아이템을 넣었을 때의 최대 가치 $dp[i - 1][w - \text{weight}] + \text{value}$ 중 더 큰 값을 선택하여 $dp[i][w]$ 에 저장

9) else:

$dp[i][w] = dp[i - 1][w]$

현재 아이템을 넣을 수 없는 경우, 이전까지의 최적값을 그대로 가져옴.

```
# 5. 선택된 아이템 역추적 - optimal solution에서 어떤 요소가 선택되었는지 파악 가능
selected_items = []
w = W
for i in range(n, 0, -1):
    if dp[i][w] != dp[i - 1][w]:
        selected_items.append(items[i - 1])
        w -= items[i - 1][0] # 무게만큼 줄이기
```

(2) 선택된 아이템 역추적: 앞서 작성한 dp 테이블을 바탕으로, 최적해를 구성한 아이템 목록을 추적하는 부분. 즉, 어떤 아이템들이 최종적으로 선택되었는지 알아내기 위한 과정

1) `selected_items = []` : 최적해에 포함된 아이템들을 저장할 리스트

2) `w = W` : 현재 배낭에 남은 무게를 의미함. 역추적의 시작점. 즉, 처음에는 최대 용량 W에서 출발하여 선택된 아이템의 무게만큼 줄여가며 추적함.

3) `for i in range(n, 0, -1):` i번 아이템부터 1번 아이템까지 역순으로 확인

4) `if dp[i][w] != dp[i - 1][w]:` i번째 아이템이 선택되었는지 확인하는 조건. 만약 $dp[i][w]$ 의 값이 $dp[i - 1][w]$ 와 다르다면, i번째 아이템을 넣음으로써 가치가 증가한 것이므로, 선택된 것임.

5) `selected_items.append(items[i - 1])`: 선택된 아이템을 `selected_items` 리스트에 추가함.

6) `w -= items[i - 1][0]` : 선택된 아이템의 무게만큼 배낭 용량 `w`를 줄임.
이후 아이템을 고려할 때는 이 줄어든 무게를 기준으로 고려함.

```
266 # 6. 선택된 아이템 출력
267 print('\n', '='*50)
268 print('\n[Dynamic Programming 결과]')
269 print("\n선택된 아이템 (정렬 후 번호, 무게, 가치):")
270 C=0
271 for item in reversed(selected_items): # 선택 순서를 앞에서부터 보기 위해 뒤집음
272     index_in_sorted = items.index(item) + 1
273     A={item[0]}
274     D=list(A)[0]
275     C+=D
276     print(f"{index_in_sorted}. ({item[0]}, {item[1]})")
277
278 end_time= time.time() - start_time
279 print("\n걸린 시간", end_time)
280 print("\n최대 무게:", C)
281 print("최대 가치:", dp[n][W])
```

(3) 선택된 아이템 출력

1) `for item in reversed(selected_items):`

선택된 아이템 리스트를 뒤집어 앞에서부터 보기 위해 작성한 부분. 즉, `selected_items`는 역추적 과정에서 뒤에서부터 채워졌기 때문에 이를 다시 원래의 순서대로 보기 위한 것.

2) `index_in_sorted = items.index(item) + 1`

: 원래의 'items'리스트에서 해당 아이템의 인덱스를 찾아 아이템 번호로 표시. +1을 하는 이유는 출력 시 인덱스가 1번부터 시작하는 것처럼 보이게 하기 위한 것.

3) `C += D`: 선택된 아이템의 무게를 누적하여 총 선택 무게 계산.

4) `print(f"{index_in_sorted}. ({item[0]}, {item[1]})")`

: 선택된 아이템의 번호, 무게, 가치를 출력.

5) `print("\n최대 무게:", C)`

: 실제로 배낭에 담은 총 무게를 출력

6) `print("최대 가치:", dp[n][W])`

: DP 테이블에서 마지막 위치에 있는 최댓값을 출력(최종 정답)

7) `end_time = time.time() - start_time` :알고리즘 실행 시간 측정

(4) Python Code 결과

[Dynamic Programming 결과]

선택된 아이템 (정렬 후 번호, 무게, 가치):

1. (1, 8)
2. (1, 7)
3. (1, 6)
4. (2, 9)
5. (2, 9)
6. (2, 7)
8. (2, 6)
9. (2, 5)

걸린 시간 0.0

최대 무게 : 13

최대 가치 : 57

3. 분기 한정법(Branch and Bound)

최적화 문제를 해결하는 데 있어, 단순한 전수 조사 방식은 비효율적이며 현실적인 문제에 적용하기 어렵다. 이러한 한계를 극복하기 위해 고안된 것이 바로 Branch and Bound 기법이다. 이 방법은 해 공간을 트리 구조로 분할(Branch)한 후, 유망하지 않은 경로를 사전에 차단(Bound)하여 탐색 효율을 높인다. Branch and Bound는 일반적인 트리 순회 탐색에 구애받지 않으며, 바운드 값을 이용하여 가능한 해 중 가장 좋은 해를 빠르게 찾도록 돕는다. 이는 단순한 백트래킹과 달리, 현재까지 발견된 최적해보다 더 나은 가능성을 가진 노드만 탐색 대상으로 고려하게 만들어, 실질적인 계산량을 획기적으로 줄일 수 있도록 도와준다.

<Python 코드 분석>

```
106 # ----- Branch and Bound 알고리즘 구현 -----
107 def Branch_and_Bound():
108     start_time = time.time()
109     global items
110     # 상태 노드 클래스 정의
111     class Node:
112         def __init__(self, level, value, weight, bound, selected):
113             self.level = level          # 현재 고려 중인 아이템 인덱스
114             self.value = value          # 현재까지 누적된 가치
115             self.weight = weight        # 현재까지 누적된 무게
116             self.bound = bound          # 이 노드에서 기대할 수 있는 최대 가치 상한
117             self.selected = selected    # 현재까지 선택한 아이템 인덱스 목록
118
119     # bound(상한값) 계산 함수
120     def bound(node):
121         if node.weight >= W:
122             return 0 # 무게 초과 시 상한은 0
123
124         profit_bound = node.value # 현재 가치에서 시작
125         j = node.level            # 다음 아이템부터 탐색
126         totweight = node.weight
127
128         # 배낭이 넘치지 않는 선까지 아이템을 greedy하게 추가
129         while j < n and totweight + items[j][0] <= W:
130             totweight += items[j][0]
131             profit_bound += items[j][1]
132             j += 1
133
134         # 그 다음 아이템을 분할하여 부분적으로 넣기 (fractional knapsack 방식)
135         if j < n:
136             weight, value, _ = items[j]
137             profit_bound += (W - totweight) * (value / weight)
138
139         return profit_bound
140
```

1) start_time=time.time()

: 시작 시간 기록(실행 시간 측정)

2) global items

: 전역 변수 'items'를 사용

3) class node~

: 상태 노드 클래스 정의.

4) def bound (node):

if node.weight>=W:

return 0

: 배낭 무게 초과시 더 이상 선택은 불가능하므로 상한은 0

5) profit_bound=node.value

j=node.level

totweight=node.weight

: 현재 가치부터 시작하여 다음 아이템부터 탐색 후에 현재까지의 무게를 구함.

6) while j<n and totweight + items[j][0] <=W:

totweight+=items[j][0]

profit bound+=items[j][1]

j+=1

: 배낭이 넘치지 않는 선까지 다음 아이템들을 추가

7) if j<n:

weight, value, _=items[j]

profit_bound += (W-totweight)*(value/weight)

: 그 다음 아이템을 분할하여 부분적으로 넣기(fractional knapsack)

```
41 # 초기화
42 max_value = 0 # 지금까지 발견된 최대 가치
43 best_selection = [] # 최적 선택 목록 (인덱스 리스트)
44
45 queue = [] # 노드를 담을 큐
46
47 # 루트 노드 생성 및 큐에 추가
48 root = Node(level=0, value=0, weight=0, bound=bound(Node(0, 0, 0, 0, [])), selected=[])
49 queue.append(root)
```

8) max_value=0

best_selection=[]

queue=[]

: 초기 최적값 설정 및 최적 선택 리스트와 탐색할 노드들을 저장하는 큐 설정

9) root = Node(level=0, value=0, weight=0, bound=bound(Node(0, 0, 0, 0, [])), selected=[])

queue.append(root)

: 루트 노드 생성 및 루트 노드를 큐에 추가.

```

151     # Branch and Bound 탐색 시작
152     while queue:
153         # 큐에서 bound가 가장 큰 노드 선택 (최대한 가능성 높은 노드)
154         max_idx = 0
155         for i in range(1, len(queue)):
156             if queue[i].bound > queue[max_idx].bound:
157                 max_idx = i
158         node = queue.pop(max_idx)
159
160         # 가지치기 조건: bound가 최대값보다 작거나, 모든 아이들을 다 본 경우
161         if node.bound <= max_value or node.level >= n:
162             continue

```

10) while queue:

main_idx=0

for i in range(1,len(queue)):

if queue[i].bound>queue[max_idx].bound:

max_idx=i

:큐에서 bound가 가장 큰 노드 선택(최대한 가능성이 높은 노드). 이 과정을 큐가 빌 때까지 반복함(BFS 탐색 구조)

11) node=queue.pop(max_idx)

:선택된 노드를 큐에서 꺼냄.

12) if node.bound <= max_value or node.level >=n:

continue

: bound가 최대값보다 작거나 모든 아이들을 다 확인한 경우에 해당 노드는 탐색하지 않고 건너뛴.

```

164     # 현재 아이템 정보 가져오기
165     weight, value, _ = items[node.level]
166
167     # 왼쪽 자식 노드: 현재 아이템을 선택한 경우
168     if node.weight + weight <= W:
169         left = Node(
170             level=node.level + 1,
171             value=node.value + value,
172             weight=node.weight + weight,
173             bound=0,
174             selected=node.selected + [node.level]
175         )
176         left.bound = bound(left) # bound 갱신
177
178     # 더 나은 해를 발견하면 저장
179     if left.value > max_value:
180         max_value = left.value
181         best_selection = left.selected
182
183     # 여전히 탐색 가치가 있으면 큐에 추가
184     if left.bound > max_value:
185         queue.append(left)

```

13) weight, value, _ = items[node.level]

: 현재 아이템 정보를 가져옴.

```
14) if node.weight + weight <= W:
    left = Node(
        level=node.level + 1,
        value=node.value + value,
        weight=node.weight + weight,
        bound=0,
        selected=node.selected + [node.level]
    )
    left.bound = bound(left)
```

: 왼쪽 자식노드는 현재 아이템을 선택한 경우로서, 노드의 다음 레벨로 진행하고, 가치와 무게는 기존에 현재 상태를 추가함. 또한, 선택목록에 현재 아이템을 추가한 후, bound값을 갱신함.

```
15) if left.value > max_value:
    max_value = left.value
    best_selection = left.selected
```

: 현재까지 발견된 것보다 더 나은 해를 발견하면 그 값을 저장함.

```
16) if left.bound >= max_value:
    queue.append(left)
```

: 여전히 탐색할 가치가 있으면 큐에 추가

```
187     # 오른쪽 자식 노드: 현재 아이템을 선택하지 않은 경우
188     right = Node(
189         level=node.level + 1,
190         value=node.value,
191         weight=node.weight,
192         bound=0,
193         selected=node.selected
194     )
195     right.bound = bound(right)
196
197     if right.bound > max_value:
198         queue.append(right)
199
200     # 선택된 아이템들의 총 무게 계산
201     total_weight = sum(items[idx][0] for idx in best_selection)
```

```
17) right = Node(
```

```

        level=node.level + 1,
        value=node.value,
        weight=node.weight,
        bound=0,
        selected=node.selected
    )
    right.bound = bound(right)

```

: 오른쪽 자식노드는 현재 아이템을 선택하지 않은 경우로서 노드의 다음 레벨로 진행하며, 가치와 무게는 기존과 변화 없음. 선택 목록 또한 변화 없음. 그 상태에서 bound값 그대로 계산

```

18) if right.bound > max_value:
    queue.append(right)

```

: 탐색할 가치가 있으면 큐에 추가

```

199     # 선택된 아이템들의 총 무게 계산
200     total_weight = sum(items[idx][0] for idx in best_selection)
201
202     # 결과 출력
203     print('\n', '='*50)
204     print('\n[Branch and Bound 결과]')
205     print("\n[선택된 아이템들] (정렬된 순서 기준 번호, 무게, 가치):")
206     for idx in best_selection:
207         w, v, _ = items[idx]
208         print(f"{idx + 1}. ({w}, {v})")
209     end_time= time.time() - start_time
210     print('\n걸린 시간', end_time)
211     print(f"\n최대 무게: {total_weight}") # 최대 무게 출력
212     print(f"최대 가치: {max_value}")

```

```

19) total_weight = sum(items[idx][0] for idx in best_selection)

```

: 최종 선택된 아이템들의 총 무게 계산

```

20) end_time= time.time() - start_time

```

: 실행 시간 측정

```

21) print(f"\n최대 무게: {total_weight}")

```

: 최대 무게 출력

22) print(f"최대 가치: {max_value}")

: 최대 가치 출력

Python 코드 실행결과

[Branch and Bound 결과]

[선택된 아이템들] (정렬된 순서 기준 번호, 무게, 가치):

1. (1, 8)
2. (1, 7)
3. (1, 6)
4. (2, 9)
5. (2, 9)
6. (2, 7)
8. (2, 6)
9. (2, 5)

걸린 시간 0.0010001659393310547

최대 무게 : 13

최대 가치 : 57

4. Greedy 알고리즘

knapsack 문제에서의 Greedy 알고리즘은 단순한 구조를 가진다.

앞서 언급한 조건 성립하에서 가방에 들어갈 물건을 선택할 때, $\frac{\text{가치}}{\text{무게}}$ 가 큰 순서대로 선택하는 과정이다.

첫 번째 물건의 무게는 가방의 용량을 넘지 못하니 선택될 수밖에 없고, 두 번째 물건을 선택할 때부터 약간의 고려 사항이 생기게 된다. 현재 가방에 담을 수 있는 무게의 한계치가 충분하여 두 번째 순서의 물건을 담을 수 있는 상황이라면 그 물건은 선택되고, 만약 가방의 무게 한계치보다 두 번째 물건이 더 무거워 가방에 들어갈 수 없다면 다음 물건으로 넘어간다. 이 과정을 계속 반복하여 가방의 용량이 다 차거나, 더 이상 담을 수 있는 물건이 존재하지 않는다면 Greedy 알고리즘은 종료된다.

Greedy 알고리즘의 장점은 단순하다는 것에 있지만, 반대로 너무 단순하기 때문에 단점도 있다. 선택 순서만을 고려하여 과정을 실행하기 때문에 최적의 조합이 선택되지 못하는 상황이 빈번하게 발생하는 문제가 발생하는 것이다.

```
def Greedy():
    start_time=time.time()
    global items
    total_weight = 0
    total_value = 0
    selected_items = []

    # 정렬된 데이터를 앞에서부터 고려하여 선택
    for weight, value, original_index in items:
        if total_weight + weight <= W:
            total_weight += weight
            total_value += value
            selected_items.append((weight, value, original_index))

    # 결과 출력
    print('\n', '*50)
    print("\n[Greedy Algorithm 결과]")
    print("\n[선택된 아이템들] (번호, 무게, 가치):")
    for idx, (weight, value, original_index) in enumerate(selected_items, start=1):
        print(f"{idx}. ({weight}, {value})")

    end_time= time.time() - start_time
    print('\n걸린 시간', end_time)
    print(f"\n최종 총 무게: {total_weight}")
    print(f"\n최종 총 가치: {total_value}")
```

[Greedy Algorithm 결과]

[선택된 아이템들] (번호, 무게, 가치):

1. (1, 8)
2. (1, 7)
3. (1, 6)
4. (2, 9)
5. (2, 9)
6. (2, 7)
7. (3, 9)

걸린 시간 0.0732419490814209

최종 총 무게: 12
최종 총 가치: 55

코드에서는 가방의 현재 무게와 가치를 고려하는 변수를 지정하고, 선택된 물건이 어떤 것인지를 파악하는 과정을 거친다.
앞서 가치의 가성비가 높은 순서대로 정렬된 물건들을 앞에서부터 고려하여 선택의 과정을 거쳐 결과를 출력한다.

5. LP Relaxation

LP Relaxation은 기존의 Knapsack 문제의 조건을 다소 완화하여 해결하는 방법이다. 기존의 Knapsack 문제는 가방 안에 물건을 담을 때, 선택지는 이 지선다 (선택하거나 선택하지 않거나)로만 존재하였다. 이는 선택의 여부가 되는 x 값이 0 또는 1로만 정의된다는 것을 의미한다.

하지만 LP Relaxation은 x 값을 정수 (0 또는 1)로만 정의하지 않고, 0과 1사이의 실수로 정의하는 방법이다. 이로 인해 물건의 선택 여부가 비교적 자유로워졌으며, 선택 여부를 고려할 때 단순한 선택을 할 수 있도록 도와준다. 다시 말하자면, x 값이 소수 값을 가질 수 있기 때문에, 가치의 가성비($\frac{\text{가치}}{\text{무게}}$)가 큰 물건부터 고려하기만 하면 된다는 사실이다. 원래의 조건이라면 여러가지 조합을 생각해야 했겠지만, 소수점을 가질 수 있는 순간 큰 물건만 선택하는 것이 가장 큰 이득이 되고, 가방은 완전히 꽉 차게 된다.


```

# 목표: maximize sum(values[i] * x[i])
# linprog는 기본이 minimization이므로, -values로 변경해서 최대화 문제로 변환
def LP_Relaxation():
    start_time = time.time()
    global items
    c = [-item[1] for item in items] # 가치를 최대화하기 위해 부호 반전
    A = [[item[0] for item in items]] # 무게에 대한 제약 조건
    b = [W] # 총 무게가 W 이하
    x_bounds = [(0, 1) for _ in items] # 각 아이템의 선택 비율 범위 (0~1)

    # linprog 호출
    res = linprog(c, A_ub=A, b_ub=b, bounds=x_bounds, method='highs')

    # 결과 출력
    print('\n', '='*50)
    print("\n[LP Relaxation 결과]")
    if res.success:
        total_value = -res.fun # 최적화 결과에서 가치
        # 총 무게 계산
        total_weight = sum(res.x[idx] * items[idx][0] for idx in range(len(res.x)))

        print("아이템별 선택 확률 (순서대로):")
        for idx, x in enumerate(res.x, start=1):
            w, v, original_index = items[idx - 1]
            print(f"item {idx}: 선택 비율 = {x:.4f}") # 아이템 선택 확률 출력
        end_time1 = time.time() - start_time
        print('\n결린 시간', end_time1)
        print(f"\n최종 총 무게: {total_weight:.2f}")
        print(f"\n최종 총 가치 (fractional): {total_value:.2f}")
    else:
        print("최적화 실패:", res.message)

```

[LP Relaxation 결과]
아이템별 선택 확률 (순서대로):
 item 1: 선택 비율 = 1.0000
 item 2: 선택 비율 = 1.0000
 item 3: 선택 비율 = 1.0000
 item 4: 선택 비율 = 1.0000
 item 5: 선택 비율 = 1.0000
 item 6: 선택 비율 = 1.0000
 item 7: 선택 비율 = 0.6667
 item 8: 선택 비율 = 1.0000
 item 9: 선택 비율 = 0.0000
 item 10: 선택 비율 = 0.0000

결린 시간 0.1520547866821289

최종 총 무게: 13.00
최종 총 가치 (fractional): 58.00

코드를 살펴보면, x의 바운드를 0과 1 사이의 값으로 지정하였고, linprog를 호출하여 문제를 해결하였다.

이 때, 출력 결과에서 item 7의 값이 아이템 8의 값보다 작은 것을 확인할 수 있다. 이론대로라면 인덱스의 오름차순으로 값이 선택되고, 마지막 값은 소수 값이 나타나야 한다.

이러한 이유는 처음 지정된 무게와 가치 값 때문이다.

[정렬된 아이템 목록] (번호, 무게, 가치):

1. (1, 8)
2. (1, 7)
3. (1, 6)
4. (2, 9)
5. (2, 9)
6. (2, 7)
7. (3, 9)
8. (2, 6)
9. (2, 5)
10. (2, 5)

7번과 8번을 보면, $\frac{\text{가치}}{\text{무게}}$ 의 값이 $\frac{9}{3} = \frac{6}{2}$ 로 같다. 이 두 값이 같기 때문에 선택 비율의 순서 차이만 있을 뿐, 결과는 같다.

6. LP Relaxation Round down

LP Relaxation 을 통해 얻은 결과는 Round Down을 이용해서 조정할 수도 있다. 각 무게와 가치는 정수 값을 가지는 것이 전제 조건이었기 때문에 이러한 과정을 시행하여 보정한다.

5. LP 결과를 Round Down 하기

```
rounded_x = np.floor(total_value) # 소수점 절삭
print('Wn', '='*50)
print("Wn[Round Down 결과]")
print(f"Wn최종 총 무게:", total_weight)
print("최종 총 가치 (fractional):", rounded_x)
```

[Round Down 결과]

최종 총 무게: 13.0

최종 총 가치 (fractional): 58.0

LP Relaxation에서 작성한 코드에 위 코드를 추가하고, 결과를 확인하였다. 위 상황에서는 소수점 버림이 발생하지 않았지만, 결론 및 고찰에서 추가로 살펴본다.

7. Matlab Intlinprog

앞서 살펴본 다양한 방법들에 더해서, 정확한 솔루션을 구하기 위해 매트랩을 사용하여 결과를 확인해 보았다. 파이썬에서 정의된 랜덤 값을 매트랩에서 정의한 후 Intlinprog 함수를 이용하여 결과를 출력하였다. (크기별로 나열하는 과정이 끝난 후의 값을 입력하였다.)

```
% 문제 데이터
weights = [1, 1, 1, 2, 2, 2, 2, 3, 2, 2, 2]; % 아이템 무게
values = [8, 7, 6, 9, 9, 9, 7, 9, 6, 5, 5]; % 아이템 가치
W = 13; % 배낭의 최대 무게

n = length(weights); % 아이템 개수

% intlinprog 기본 형식은 "minimize" 이므로, maximize를 위해 부호를 반대로
f = -values; % maximize value -> minimize (-value)
intcon = 1:n; % 모든 변수를 정수형 (binary)
A = weights; % 무게 제약
b = W; % 최대 무게

% intlinprog 실행
opts = optimoptions('intlinprog', 'Display', 'off');
[x, fval] = intlinprog(f, intcon, A, b, [], [], zeros(n,1), ones(n,1), opts);

% 선택된 아이템 출력
disp(['선택된 아이템들 (정렬된 순서 기준 번호, 무게, 가치):']);
selected_items = find(x == 1); % 선택된 아이템들의 인덱스

% 선택된 아이템을 번호순으로 정렬하여 출력
for i = 1:length(selected_items)
    idx = selected_items(i);
    fprintf('%d. (%d, %d)\n', idx, weights(idx), values(idx));
end

% 최대 무게와 최대 가치 출력
total_weight = sum(weights(selected_items)); % 선택된 아이템들의 총 무게
total_value = -fval; % 최대 가치 (최소화한 값의 부호 반전)
disp(['최대 무게: ', num2str(total_weight)]);
disp(['최대 가치: ', num2str(total_value)]);
```

- `f = -values;`

: 최대화 문제(가치의 합을 최대화)를 최소화 문제로 바꾸기 위해 `-values`로 설정

(이유: MATLAB의 `intlinprog`는 기본적으로 최소화 문제만 다루므로 부호를 바꿔야 함)

- `A = weights';`

`b = W;`

: 부등식 제약의 계수 행렬을 정의. $Ax \leq b$ 형식을 따르며, 여기서 총무게 $\leq W$ 라는 제약을 설정. 여기서 x 는 아이템 선택 여부(0 or 1)을 나타냄. b 는 부등식 제약의 우변 값, 즉 배낭의 최대 무게를 설정.

- `intcon = 1:num_items;`

: 정수형(이진형) 변수의 인덱스를 지정. 여기서 모든 변수($x(i)$)가 정수(0 또는 1)로 제한되어야 하므로 전체 인덱스를 넣음.

- `lb = zeros(num_items, 1);`

: 변수의 하한값을 0으로 설정. 어떤 아이템도 선택하지 않을 수 있음.

- `ub = ones(num_items, 1);`

: 변수의 상한값을 1로 설정. 각 아이템은 최대 한 번만 선택 가능.

- `[x, fval] = intlinprog(f, intcon, A, b, [], [], lb, ub);`

: 이진 정수계획법 문제 풀이.

- `selected_items = find(x > 0.5);`

: x 에서 0.5보다 큰 값(즉, 1)을 갖는 인덱스를 찾아 선택된 아이템의 번호를 저장

- `total_value = -fval;`

: 앞서 목적 함수를 음수로 바꿨기 때문에, 다시 부호를 원래대로 돌려 총 가치를 출력.

[선택된 아이템들] (정렬된 순서 기준 번호, 무게, 가치):

1. (1, 8)
2. (1, 7)
3. (1, 6)
4. (2, 9)
5. (2, 9)
6. (2, 7)
8. (2, 6)
10. (2, 5)
최대 무게: 13
최대 가치: 57

(index 번호가 다른 것은 같은 수치를 가진 것 때문에 다른 것임.)

8. 결론 및 고찰

앞서 실행한 방법들을 비교해보자.

[Dynamic Programming 결과]

[선택된 아이템들] (번호, (무게, 가치)):

1. (1, 8)
2. (1, 7)
3. (1, 6)
4. (2, 9)
5. (2, 9)
6. (2, 7)
8. (2, 6)
9. (2, 5)

걸린 시간 0.08648514747619629

최대 무게: 13
최대 가치: 57

[Branch and Bound 결과]

[선택된 아이템들] (번호, (무게, 가치)):

1. (1, 8)
2. (1, 7)
3. (1, 6)
4. (2, 9)
5. (2, 9)
6. (2, 7)
8. (2, 6)
9. (2, 5)

걸린 시간 0.05836319923400879

최대 무게: 13
최대 가치: 57

우선 Dynamic과 Branch and Bound 방법은 모든 상황을 전부 고려하는 방법으로, 가장 최적화된 솔루션을 제공한다. 이는 Matlab intlinprog를 통해서도 확인할 수 있다.

[Greedy Algorithm 결과]

[선택된 아이템들] (번호, (무게, 가치)):

1. (1, 8)
2. (1, 7)
3. (1, 6)
4. (2, 9)
5. (2, 9)
6. (2, 7)
7. (3, 9)

걸린 시간 0.07737946510314941

최종 총 무게: 12

최종 총 가치: 55

Greedy 알고리즘은 선택 방법을 단순화하여 결과를 도출하는 방법으로, Dynamic과 Branch and Bound 방법보다는 총 가치 값이 작게 나타났다.

[LP Relaxation 결과]

아이템별 선택 확률 (순서대로):

- item 1: 선택 비율 = 1.0000
- item 2: 선택 비율 = 1.0000
- item 3: 선택 비율 = 1.0000
- item 4: 선택 비율 = 1.0000
- item 5: 선택 비율 = 1.0000
- item 6: 선택 비율 = 1.0000
- item 7: 선택 비율 = 0.6667
- item 8: 선택 비율 = 1.0000
- item 9: 선택 비율 = 0.0000
- item 10: 선택 비율 = 0.0000

걸린 시간 0.1520547866821289

최종 총 무게: 13.00

최종 총 가치 (fractional): 58.00

[Round Down 결과]

최종 총 무게: 13.0

최종 총 가치 (fractional): 58.0

LP Relaxation 방법은 물건을 선택하는 상황을 다소 비현실적으로 가정하지만, 가방의 무게 용량을 항상 꽉 채울 수 있으며, 그 중에서도 가장 높은 가치를 지니는 상황을 도출하게 만들어준다.

이에 더해서 Round Down은 LP Relaxation 방법을 기반으로 도출된 결과에서 소수점은 모두 버리고 정수 값만 취하는 방법이다. LP Relaxation으로 구한 최적해를 조건에 맞도록 조정하였기 때문에 최적해를 가지는 상황은 많지 않다.

이러한 4가지 방법을 통한 결과에서, 각 가방이 지니고 있는 가치 정도를 비교해보면 다음과 같다.

Greedy ≤ Dynamic = Branch and Bound ≤ LP Relaxation Round down ≤ LP Relaxation

(55 ≤ 57 ≤ 58 ≤ 58)

Dynamic과 Branch and Bound 보다 LP Relaxation이 더 큰 값을 가지는 것은 LP Relaxation의 해는 소수를 포함하기 때문이며, Greedy가 Dynamic과 Branch and bound 보다 더 작은 값을 가지는 것은 비교적 단순한 과정을 거쳤기 때문이다.

<추가 설명>

앞서 지정된 수치 이외에도 몇 가지 상황을 추가로 가정하여 살펴보도록 하자.

상황 1.

- [정렬된 아이템 목록] (번호, 무게, 가치):
- 1. (3, 13)
 - 2. (4, 12)
 - 3. (5, 11)
 - 4. (7, 13)
 - 5. (8, 12)
 - 6. (11, 13)
 - 7. (10, 11)
 - 8. (13, 14)
 - 9. (15, 14)
 - 10. (13, 12)

위와 같은 상황일 때,

[Dynamic Programming 결과]

최대 무게: 48
최대 가치: 85

=====

[Branch and Bound 결과]

최대 무게: 48
최대 가치: 85

=====

[Greedy Algorithm 결과]

최종 총 무게: 48
최종 총 가치: 85

=====

[LP Relaxation 결과]

최종 총 무게: 48.00
최종 총 가치 (fractional): 85.00

=====

[Round Down 결과]

최종 총 무게: 48
최종 총 가치: 85

Greedy=Dynamic=Branch and Bound=LP Relaxation Round down=LP Relaxation

모든 결과 값이 같은 값으로 나오는 상황도 존재한다. (<가 아닌 ≤인 이유)

=====

상황 2.

[정렬된 아이템 목록] (번호, 무게, 가치):

1. (1, 14)
2. (1, 12)
3. (1, 11)
4. (2, 11)
5. (3, 14)
6. (3, 13)
7. (3, 13)
8. (3, 13)
9. (4, 12)
10. (4, 12)

위와 같은 상황일 때,

[Dynamic Programming 결과]

최대 무게: 15
최대 가치: 90

=====

[Branch and Bound 결과]

최대 무게: 15
최대 가치: 90

[Greedy Algorithm 결과]

최종 총 무게: 14
최종 총 가치: 88

=====

[LP Relaxation 결과]

최종 총 무게: 16.00
최종 총 가치 (fractional): 96.67

=====

[Round Down 결과]

최종 총 무게: 16.0
최종 총 가치 (fractional): 96.0

$\text{Greedy} \leq \text{Dynamic} = \text{Branch and Bound} \leq \text{LP Relaxation} \text{ Round down} < \text{LP Relaxation}$ 인 상황이 발생하기도 한다.

(기존과는 다르게 소수점이 발생하여 실질적 Round Down 과정을 거침)